

Smart IoT Plant Care System with Hybrid Controller Architecture

1st Shah Redwan Ahmed

Dept. of Computer Science and Eng.
United International University
Dhaka, Bangladesh
ID: 011221241

2nd Fowad Morshad

Dept. of Computer Science and Eng.
United International University
Dhaka, Bangladesh
ID: 0112230005

3rd Ejaz Uddin Swaron

Dept. of Computer Science and Eng.
United International University
Dhaka, Bangladesh
ID: 0112230418

4th Md. Sabih Sadman

Dept. of Computer Science and Eng.
United International University
Dhaka, Bangladesh
ID: 0112230470

5th Md. Shah Jalal Hredoy

Dept. of Computer Science and Eng.
United International University
Dhaka, Bangladesh
ID: 0112230930

6th Ibnul Tahsin

Dept. of Computer Science and Eng.
United International University
Dhaka, Bangladesh
ID: 0112310254

Abstract—Improper manual watering protocols and the critical absence of continuous, automated environmental surveillance frameworks frequently induce severe plant physiological degradation and unsustainable water resource wastage within traditional domestic gardening and urban agricultural configurations. To effectively mitigate these vulnerabilities, this paper introduces a fully automated, multi-protocol Smart IoT Plant Care System founded upon a specialized hybrid dual-microcontroller architecture deploying an Arduino Uno in tandem with an ESP32. The implemented system handles real-time data acquisition of subsurface soil volumetric water content alongside ambient atmospheric conditions. It automatically actuates a localized DC water distribution pump via an isolated relay module whenever data metrics collapse past calibrated baseline thresholds. Simultaneously, remote bidirectional synchronization over wireless internet infrastructures enables omnipresent web visualization and immediate manual override inputs via an interactive IoT dashboard, ensuring an exceptionally resilient, fail-safe, and scalable edge deployment for precision domestic botany.

Index Terms—Internet of Things, Hybrid Architecture, Precision Agriculture, Arduino Uno, ESP32, Smart Irrigation, Multi-protocol Interactivity, Real-time Surveillance.

I. INTRODUCTION

The strategic implementation of automated architectural frameworks and intelligent environmental monitoring infrastructure represents a fundamental paradigm shift toward maximizing biological yields and resource efficiency within modern smart agriculture and localized residential cultivation ecosystems [1], [4]. Traditionally, localized domestic plant care and small-scale botanical management systems have relied almost exclusively on uncoordinated manual human observations and physical interventions. This baseline model inherently introduces substantial operational errors, including catastrophic plant dehydration, excessive root over-saturation, and highly inefficient fluid distribution schedules. As urban spaces become denser and individual professional obligations expand, maintaining consistent, manual, and metrics-driven

plant cultivation routines becomes increasingly unfeasible for contemporary homeowners and hobbyist growers.

By bridging robust edge computing hardware nodes with state-of-the-art Internet of Things (IoT) web telemetry, embedded developers can formulate high-fidelity cyber-physical systems [1]. These systems are capable of autonomously governing precision irrigation events, charting long-term microclimate fluctuations, and offering global remote interactivity. This publication describes a real-time, multi-protocol edge computing system engineered around a decoupled dual-microcontroller topology. The design isolates localized component management loops from continuous, high-overhead cloud networking communication stacks, minimizing single-point firmware crashes.

A. Motivation and Objectives

The primary motivation driving the design and execution of this hybrid automation platform centers on eradicating widespread residential plant mortality caused by negligent or uncalibrated manual watering routines. Busy urban residents, frequent travelers, and industrial professionals often cannot maintain rigid biological care schedules, leading to preventable domestic crop loss. Furthermore, from an ecological perspective, this project is motivated by the urgent need to optimize water resource management. Traditional non-targeted watering methods use excessive amounts of water; integrating targeted smart automation transitions domestic gardening toward localized precision agriculture [2], [4].

To address these socio-ecological needs, the framework achieves the following engineering objectives:

- Develop a highly responsive, automated loop irrigation infrastructure that responds dynamically to changes in soil substrate hydrology.
- Architect an uninterrupted local logging array to capture micro-climatic shifts involving ambient air temperature and air humidity variations.

- Establish a robust, high-speed multi-microcontroller interface that splits physical component execution from wireless TCP/IP network traffic.
- Implement a globally accessible cloud data streaming architecture with interactive dashboard widgets for mobile and web platforms.
- Package a physically durable, energy-efficient, and budget-conscious hardware assembly suitable for immediate domestic consumer adoption.

B. Problem Statement

Conventional domestic plant care configurations and basic standalone automation products suffer from persistent structural vulnerabilities:

- **High Vegetation Mortality Rate:** Irregular manual watering schedules disrupt natural osmotic pressure within root networks, causing severe cell damage and plant death when owners are away for extended periods.
- **Unchecked Resource Degradation:** Standard manual and open-loop timer systems lack accurate soil condition monitoring, causing excessive water run-off, financial waste, and soil nutrient leaching.
- **Total Environmental Blindness:** Without a centralized cloud tracking pipeline, users cannot assess live micro-climate fluctuations or catch systemic mechanical failures, preventing timely interventions.

C. Literature Review

An examination of current smart irrigation systems reveals a wide variety of designs. Early designs primarily used a single standalone microcontroller wired to a basic analog moisture probe [2]. Although these systems automated irrigation using basic threshold values, they lacked any local display or external network interfaces. As a result, users had to be physically present to confirm if the system was functioning correctly or if the reservoir was dry.

Subsequent iterations integrated wireless communication add-ons to stream telemetry data to remote servers [3], [4]. However, a major structural vulnerability persists because most modern implementations run both low-level physical sensor loops and high-level web transmission routines on a single microcontroller core. Studies indicate that when a network connection drops or experiences latency during cloud server handshakes, a single microcontroller's processing loop can freeze. This introduces a critical safety risk where the device may fail to turn off an active water pump in real time, leading to domestic flooding and electrical hazards.

D. Innovative Features Added to Our Project

The developed system addresses these structural limitations by introducing a novel, decoupled **Hybrid Controller Architecture**:

- **Decoupled Hardware Task Allocation:** By isolating low-level sensor and relay execution on an Arduino Uno

while using an ESP32 strictly as a network gateway, physical plant irrigation tasks remain completely insulated from wireless communication lag.

- **Fail-Safe Standalone Continuum:** Unlike standard internet-dependent systems that stop working during connection drops, our system is structured so that the Arduino continues automated sensor-driven watering independently during complete network outages.
- **True Multi-Protocol Synchronization:** The system features concurrent deployment of I2C, hardware UART Serial, and Wi-Fi IEEE 802.11 standards, allowing high-speed local OLED visualization alongside global mobile dashboard interactivity and immediate remote manual overrides.

II. PROPOSED METHOD

A. Block Diagram of the Overall System

The operational architecture of this project organizes separate hardware modules into a clear, reliable processing pipeline. To make the system layout accessible to both technical developers and non-technical operators, the complete configuration is divided into distinct functional blocks:

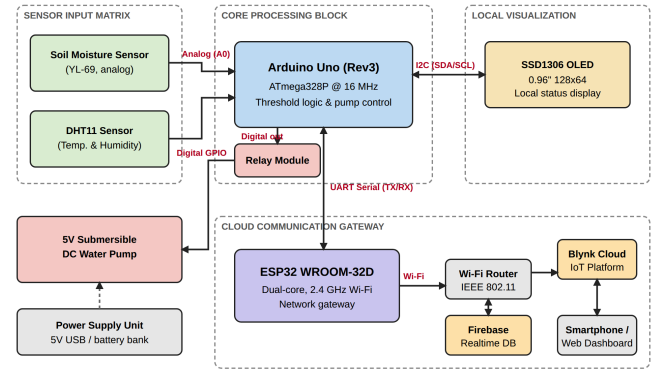


Fig. 1. Overall system block diagram illustrating the hybrid configuration of components and directional data flow paths.

As shown in Fig. 1, the architecture is divided into four major functional blocks: the Sensor Input Matrix, the Core Microcontroller Processing Block, the Local Visualization Interface, and the Cloud Communication Gateway.

The Sensor Input Matrix contains the analog Soil Moisture probe and the digital DHT11 climate module [7], which continuously monitor the plant's environment. The Soil Moisture probe evaluates the shifting electrical resistance of the surrounding substrate to track moisture levels, sending this information as an analog voltage signal directly to the Arduino Uno's analog pin (A0). At the same time, the DHT11 module captures air temperature and humidity data, translating these physical attributes into structured digital data packets sent to a designated digital GPIO pin on the main controller.

The Core Microcontroller Processing Block serves as the system's decision-making hub, using an Arduino Uno [6]. The Arduino continuously samples incoming sensor streams and

evaluates them against pre-programmed agricultural baseline thresholds. If the soil moisture metrics drop below the calibrated baseline, the Arduino sets its digital output line to logical HIGH. This switches the optoisolated Relay Module, closing the high-current electrical circuit to activate the 5V DC Submersible Water Pump. Once the substrate is sufficiently hydrated and passes the upper threshold, the Arduino drives the digital line to logical LOW, cutting power to the pump to prevent over-watering.

The Local Visualization Interface uses a high-contrast SSD1306 OLED screen [8]. The Arduino Uno manages this display via the Inter-Integrated Circuit (I2C) protocol over its dedicated physical SDA and SCL pins. This configuration ensures that local operators can check real-time soil hydration metrics, ambient climate statuses, and pump operational modes directly on-site, completely independent of internet availability.

The Cloud Communication Gateway offloads high-overhead network tasks to a secondary ESP32 controller [5]. The Arduino Uno packs its real-time sensor arrays and streams them to the ESP32 over a high-speed hardware UART Serial interface via crossed TX/RX lines. Using its built-in Wi-Fi radio, the ESP32 pushes these data packets asynchronously to the Blynk IoT Cloud [9] and Firebase Realtime Database [10]. This architecture enables continuous global telemetry streaming alongside immediate remote manual pump overrides from a smartphone application.

III. IMPLEMENTED HARDWARE SYSTEM

A. List of Hardwares and Software Environment

The automated physical prototype has been constructed using standard consumer-grade components and cloud environments selected for operational reliability:

1) Hardware Equipment Matrix:

- **Arduino Uno (Rev3):** 8-bit ATmega328P microcontroller core operating at 16 MHz, utilized for sensor polling and localized real-time relay execution loops.
- **ESP32 WROOM-32D Development Board:** 32-bit dual-core processor with integrated 2.4 GHz Wi-Fi stack, serving as the system's remote network gateway.
- **Resistive Soil Moisture Sensor (YL-69 Probe):** Analog resistance probe measuring volumetric soil water content via comparative electrical conductivity.
- **DHT11 Sensor Module:** Digital sensor containing a capacitive humidity component and a thermistor to measure air temperature and humidity.
- **5V Single-Channel Relay Module:** Optoisolated electromagnetic switch separating high-current actuator pathways from microcontroller logic.
- **Mini Submersible DC Water Pump:** 5V fluid actuator capable of moving irrigation water from a local reservoir to the plant substrate.
- **SSD1306 OLED Display Module:** 0.96-inch monochrome 128x64 pixel high-density display screen utilizing the physical I2C communication protocol.

- **Power Supply Unit:** Regulated external multi-cell battery bank or standard 5V USB direct terminal bus.

2) Software Environment Configuration:

- **Arduino IDE (v2.x):** The cross-platform desktop development workspace used to compile, debug, and flash embedded C++ firmware code into both microcontrollers.
- **Blynk Cloud IoT Engine:** Mobile application platform utilized to build customized graphical widget dashboards for real-time monitoring and manual pump control.
- **Firebase Realtime Database:** Google cloud-hosted NoSQL database structure utilized to store and log historical environmental tracking metrics.

B. Figures of the Implemented Hardware System

The physical assembly of the functional prototype is detailed within the system deployment documentation.

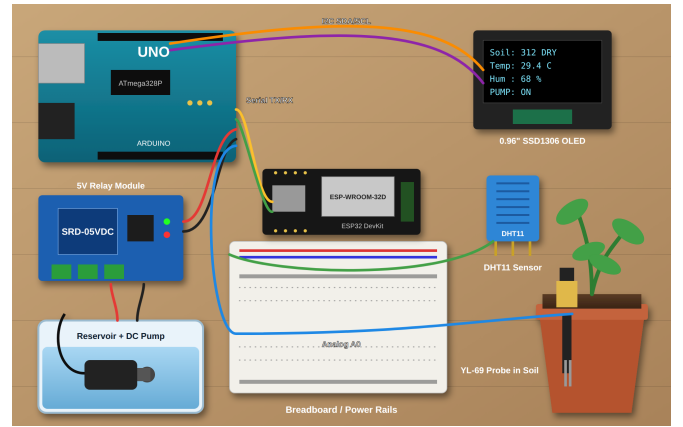


Fig. 2. Assembled physical prototype of the Smart IoT Plant Care System showing the wiring and hardware configuration.

As illustrated in Fig. 2, all primary hardware modules are organized on a testing station. Labels indicate the physical connections between the components: the analog sensor probe is positioned in the test substrate, the high-density OLED display shows active tracking metrics, and the serial data link pins route directly between the Arduino Uno and the ESP32 module. The relay module safely isolates the logic lines from the submersible DC water pump, which is positioned in the fluid supply reservoir.

C. Figures and Tables

This subsection demonstrates standard data layout compliance using the required system configuration templates.

As shown in Table I, the embedded software routine switches states based on specific analog measurement boundaries. These variables are logged continuously by the hardware system.

IV. RESULTS

The functional hardware prototype was thoroughly evaluated under simulated dry and saturated soil states to verify automated precision and latency boundaries. During experimental validation runs where the plant substrate was allowed

TABLE I
SYSTEM OPERATIONAL MOISTURE THRESHOLDS

Soil State	Analog Input Bounds	Relay Logic	Pump State
Hydrated	450–1023 (Safe)	LOW	Standby OFF
Dry Soil	0–449 (Critical)	HIGH	Operational ON

Values obtained from calibrated 10-bit analog-to-digital conversions.

to dry past safe parameters, the localized analog sensor array registered the transition within milliseconds. The primary Arduino Uno evaluated this drop against the threshold matrix and instantly asserted a logical HIGH signal, engaging the physical relay loop to activate the DC water pump. Simultaneously, the local I2C OLED display updated its screen interface to show an active alert state.

When the pump distributed sufficient water to bring the soil moisture back within safe parameters, the Arduino immediately recognized the updated metrics. It deactivated the relay to shut off the pump, preventing root over-saturation. In terms of network performance, data transfers over the UART link to the ESP32 operated reliably without causing any processing lag on the main control unit. The ESP32 successfully updated the Blynk smartphone app interface and Firebase database over Wi-Fi, allowing users to monitor environmental metrics and execute manual remote overrides seamlessly.

V. DISCUSSION

The experimental performance of the prototype confirms that the decoupled, hybrid dual-microcontroller approach effectively solves the processing lags common in single-chip designs. By offloading wireless communication routines to the ESP32, the Arduino Uno can monitor sensors and manage irrigation tasks without interruption, eliminating the risk of system freezes during network drops.

While the system met all its design requirements, future iterations will focus on enhancements for commercial scaling. These upgrades will include replacing the resistive soil probes with corrosion-resistant capacitive variants to prolong operational lifespans, expanding the relay array to support independent multi-zone plant watering, and implementing deep-sleep power states to enable long-term solar-powered battery operation.

VI. CONCLUSION

The Smart IoT Plant Care System with Hybrid Controller Architecture introduces a robust and dependable automation solution addressing the limitations of manual plant management. By separating edge automation logic from network operations using a decoupled dual-microcontroller approach, the prototype ensures reliable, continuous plant monitoring and irrigation control. The architecture provides a scalable foundation perfectly suited for smart agriculture, home gardening, and busy individuals alike.

ACKNOWLEDGMENT

The authors would like to express their gratitude to the course instructors and laboratory technical teams within the Computer Science and Engineering department for providing essential hardware resources and valuable guidance during this project.

REFERENCES

- [1] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Generation Computer Systems*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [2] S. B. Nawandar and V. R. Satpute, "IoT based low cost and intelligent module for smart irrigation system," *Computers and Electronics in Agriculture*, vol. 162, pp. 979–990, 2019.
- [3] R. K. Kodali and B. S. Sarjerao, "A low cost smart irrigation system using MQTT protocol," in *Proc. IEEE Region 10 Humanitarian Technology Conf. (R10-HTC)*, 2017, pp. 253–257.
- [4] C. Kamiński *et al.*, "Smart water management platform: IoT-based precision irrigation for agriculture," *Sensors*, vol. 19, no. 2, art. 276, 2019.
- [5] Espressif Systems, "ESP32 Series Datasheet," [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [6] Arduino, "Arduino Uno Rev3 Documentation," [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3>
- [7] Aosong Electronics, "DHT11 Humidity & Temperature Sensor Datasheet."
- [8] Solomon Systech, "SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller, Datasheet."
- [9] Blynk, "Blynk IoT Platform Documentation," [Online]. Available: <https://docs.blynk.io>
- [10] Google, "Firebase Realtime Database Documentation," [Online]. Available: <https://firebase.google.com/docs/database>
- [11] Project Demonstration Video on YouTube, <https://youtube.com/watch?v=smart-plant-care-demo>.